دليل شامل 🏗️

Clean Architecture & Clean Code

من المبادئ إلى التطبيق العملي في Flutter & Dart

الجزء الأول: Clean Architecture

١ - ما هي Clean Architecture؟

Clean Architecture هي نهج معماري طرحه Robert C. Martin (المعروف بـ Uncle Bob) عام 2012، وهدفها الأساسي هو فصل الاهتمامات (Separation of Concerns) بشكل واضح وصارم داخل قاعدة الكود.

> 🎯 المبدأ الجوهري: الكود الداخلي (Business Logic) لا يعتمد أبداً على أي تفاصيل خارجية كـ Frameworks أو Databases أو UI.

الفكرة مبنية على قانون الاعتمادية (Dependency Rule):

- الاعتماديات تسير دائماً من الخارج للداخل فقط.
- الطبقات الداخلية لا تعرف شيئاً عن الطبقات الخارجية.
- كلما اتجهنا للداخل، كلما زادت أهمية وثبات الكود.

٢ - طبقات Clean Architecture

تتكون البنية من دوائر متداخلة (Concentric Circles)، كل دائرة تمثل طبقة:

الطبقة	المحتوى	الاعتمادية	مثال في Flutter
Entities (المركز)	Business Objects — قواعد العمل الأساسية	لا تعتمد على شيء	User, Order, Product
Use Cases (Application)	Application Logic — العمليات التجارية	تعتمد على Entities فقط	LoginUseCase — GetProductsUseCase
Interface Adapters	Controllers, Presenters, Repository Interfaces	تعتمد على Use Cases	UserRepository (abstract) — ProductCubit

الطبقة	المحتوى	الاعتمادية	مثال في Flutter
Frameworks & Drivers (الخارج)	UI, Database, API, External Libraries	تعتمد على Interface Adapters	Flutter Widgets — Firebase, Dio, SQLite

⚠️ الدوائر ليست ثابتة بالضبط على أربع — يمكنك إضافة طبقات إضافية حسب حجم المشروع. المهم هو الالتزام بـ Dependency Rule.

◆ أ) طبقة Entities

هي الطبقة الأعمق والأكثر استقراراً. تحتوي على Business Objects والقواعد التجارية الأساسية. لا تعرف شيئاً عن Flutter أو أي Framework خارجي.

```
// ✅ Entity لا توجد أي imports خارجية — نظيفة
class User {
  final String id;
  final String name;
  final String email;

  const User({required this.id, required this.name, required this.email});

  // Business Rule: التحقق من صحة البريد الإلكتروني
  bool get isValidEmail => email.contains('@') && email.contains('.');
}
```

◆ ب) طبقة Use Cases

تحتوي على Application Business Rules. كل Use Case يمثل عملية واحدة فقط (Single Responsibility) وتعتمد على Interfaces وليس Implementations.

```
// ✅ Use Case نظيف
class LoginUseCase {
  final AuthRepository repository; // interface وليس implementation

  LoginUseCase(this.repository);

  Future<Either<Failure, User>> call(LoginParams params) {
    return repository.login(params.email, params.password);
  }
}
```

◆ ج) طبقة Interface Adapters

هذه الطبقة تحوّل البيانات بين الشكل الملائم لـ Use Cases وبين الشكل الملائم للخارج (UI, DB). تحتوي على Repository Implementations وCubits وData Models.

```
// ✅ Repository Implementation في هذه الطبقة
class AuthRepositoryImpl implements AuthRepository {
  final RemoteDataSource remoteDataSource;
  final LocalDataSource localDataSource;

  @override
  Future<Either<Failure, User>> login(String email, String password) async {
```

```
    try {
      final userModel = await remoteDataSource.login(email, password);
      return Right(userModel.toEntity()); // تحويل Model إلى Entity
    } catch (e) {
      return Left(ServerFailure(e.toString()));
    }
  }
}
```

◆ د) طبقة Frameworks & Drivers

الطبقة الخارجية وتشمل كل شيء خارجي: Flutter Widgets، مكتبات HTTP كـ Dio، قواعد البيانات المحلية، وأي External Service.

> ✅ هذه الطبقة هي الأسهل في الاستبدال — إذا أردت تغيير Firebase بـ Supabase مثلاً، فالتغيير يحدث هنا فقط ولا يمس بقية الكود.

٣ - هيكل المجلدات في Flutter

هيكل نموذجي لمشروع Flutter يطبق Clean Architecture:

```
lib/
├── core/                          # مشترك بين كل الـ Features
│   ├── error/                     # Failures & Exceptions
│   └── usecases/                  # Base UseCase class
│
├── features/                      # كل Feature مستقل بذاته
│   └── auth/                      # مثال: ميزة التسجيل
│       ├── domain/                # الطبقة الداخلية
│       │   ├── entities/          # User, Session
│       │   ├── repositories/      # AuthRepository (abstract)
│       │   └── usecases/          # LoginUseCase, LogoutUseCase
│       │
│       ├── data/                  # طبقة Interface Adapters
│       │   ├── models/            # UserModel (extends User)
│       │   ├── datasources/       # Remote & Local DataSource
│       │   └── repositories/      # AuthRepositoryImpl
│       │
│       └── presentation/          # الطبقة الخارجية
│           ├── bloc/              # AuthCubit / AuthBloc
│           ├── pages/             # LoginPage, RegisterPage
│           └── widgets/           # LoginForm, AuthButton
│
└── injection_container.dart       # Dependency Injection
```

٤ - فوائد Clean Architecture

- Testability عالية: كل طبقة قابلة للاختبار بشكل منفصل باستخدام Mocks.
- Maintainability: أي تغيير في طبقة لا يؤثر على الطبقات الأخرى.
- Scalability: سهولة إضافة Features جديدة بدون تعديل الكود الموجود.
- Framework Independence: يمكن نظرياً نقل Business Logic لأي Framework.
- Database Independence: استبدال DB بسهولة دون تأثير على Business Logic.

٥ - متى لا تستخدم Clean Architecture؟

⚠️ Clean Architecture ليست مناسبة لكل مشروع — تزيد من حجم Boilerplate Code بشكل كبير.

- المشاريع الصغيرة أو الـ POC: الـ Overhead أكبر من الفائدة.
- الـ Deadlines الضيقة: تحتاج وقتاً في الإعداد الأولي.
- الفريق الصغير الجديد على المفهوم: يحتاج تدريباً مسبقاً.

الجزء الثاني: Clean Code

١ - ما هو Clean Code؟

Clean Code مصطلح شهّره Robert C. Martin في كتابه الشهير (2008). لكنه ليس نهجاً معمارياً — بل هو مجموعة مبادئ وممارسات تجعل الكود سهل القراءة والتعديل والاختبار.

> 📖 "Clean code reads like well-written prose." — Grady Booch الكود النظيف يُقرأ مثل النثر المكتوب بعناية.

٢ - أسماء ذات معنى (Meaningful Names)

اختيار الأسماء الصحيحة هو أهم جانب في Clean Code:

```
// ❌ اسم مبهم – ماذا يعني d؟
int d = 7;

// ✅ اسم واضح يعبر عن القصد
int daysUntilDeadline = 7;
```

```
// ❌ اسم الدالة لا يوضح ماذا تفعل
void process(User u) { ... }

// ✅ اسم يصف العملية بوضوح
void sendWelcomeEmailToUser(User user) { ... }
```

- استخدم أسماء قابلة للنطق: customerRecord أفضل من cstRcd.
- تجنب الأحرف المنفردة: i, j مقبولة في الـ loops فقط.
- Classes تبدأ بحرف كبير: UserRepository وليس userRepository.
- Booleans تبدأ بـ is/has/can: مثل isLoggedIn أو hasPermission.

٣ - الدوال (Functions)

◆ قاعدة: الدالة تفعل شيئاً واحداً فقط (Single Responsibility)

إذا استطعت وصف الدالة بـ "وأيضاً" — فهي تفعل أكثر من شيء وتحتاج تقسيم.

```
// ❌ دالة تفعل أكثر من شيء
Future<void> loginAndSaveUser(String email, String pass) async {
  final user = await api.login(email, pass);      // مهمة 1: تسجيل الدخول
  await db.save(user);                            // مهمة 2: حفظ البيانات
  navigator.pushReplacement(HomeRoute());         // مهمة 3: التنقل!
}
```

```
// ✅ فصل المسؤوليات – كل دالة مهمة واحدة
Future<User> login(String email, String pass) =>
    authRepository.login(email, pass);

Future<void> saveUser(User user) => userRepository.save(user);
```

◆ قاعدة: عدد البارامترات

- صفر بارامترات: الأفضل دائماً.
- واحد أو اثنان: مقبول جداً.
- ثلاثة: استخدمه بحذر.
- أكثر من ثلاثة: استخدم Object أو Class بدلاً منه.

```
// ❌ كثير من البارامترات – صعب القراءة والاختبار
void createUser(String name, String email, String phone, int age) {...}

// ✅ استخدام Object – أوضح وأسهل في الاختبار
void createUser(CreateUserParams params) {...}
```

٤ - التعليقات (Comments)

💬 الكود السيء لا يُحسّن بالتعليقات — يُحسّن بإعادة الكتابة. Robert C. Martin

التعليقات الجيدة — اشرح القرار (Why) وليس ماذا يفعل الكود (What):

- شرح القرارات الغير واضحة أو المنطق المعقد.
- TODO & FIXME مع ذكر السبب.
- توثيق Public APIs.

التعليقات السيئة التي يجب تجنبها:

- تعليق يعيد ما يقوله الكود بالضبط بدون إضافة.
- تعليقات قديمة لا تتطابق مع الكود الحالي.
- كود معلّق بدلاً من حذفه نهائياً.

```
// ❌ تعليق زائد – الكود يقول نفس الشيء
int age = 25; // نعيّن قيمة 25 للمتغير age

// ✅ تعليق يشرح القرار وليس الكود
// نضيف delay لتجنب rate limiting عند الـ API
await Future.delayed(const Duration(milliseconds: 500));
```

٥ - معالجة الأخطاء (Error Handling)

- لا تُعيد null — استخدم Either أو Result types.
- لا تَبتلع الأخطاء في catch فارغة بدون معالجة.
-
- الأخطاء جزء من Business Logic — عاملها كذلك.

```
// ❌ ابتلاع الخطأ – لا أحد يعلم ماذا حدث
try {
  await loadData();
} catch (e) {}

// ✅ معالجة واضحة ومُعبّرة
try {
  await loadData();
} on NetworkException catch (e) {
  emit(DataState.networkError(e.message));
} on ServerException catch (e) {
  emit(DataState.serverError(e.statusCode));
}
```

٦ - مبادئ DRY و KISS

DRY — Don't Repeat Yourself ◆

كل قطعة معرفة أو Business Rule يجب أن تكون في مكان واحد فقط. التكرار يعني أنك عند التغيير ستحتاج لتعديل أكثر من مكان وهذا مصدر للـ Bugs.

KISS — Keep It Simple, Stupid ◆

الكود البسيط أفضل من الكود الذكي. اكتب الكود الذي يفهمه زميلك الجديد في الفريق — وليس الكود الذي يُثير إعجابك وحدك.

> Debugging is twice as hard as writing the code. If you write the code as cleverly as" 🧠 possible, you are not smart enough to debug it." — Brian Kernighan

الجزء الثالث: الفرق بينهما

١ - جدول المقارنة الشاملة

المحور	Clean Architecture	Clean Code
التعريف	نهج معماري لتنظيم المشروع إلى طبقات	ممارسات لكتابة كود مقروء وقابل للصيانة
المستوى	Macro — على مستوى المشروع كله	Micro — على مستوى الكلاس والدالة والسطر
السؤال الذي يجيب عليه	أين أضع هذا الكود؟ كيف أنظم المشروع؟	كيف أكتب هذه الدالة؟ ما الاسم المناسب؟
متى تطبقه؟	عند بداية المشروع أو إعادة هيكلته	في كل سطر كود تكتبه طوال اليوم
التأثير	يحدد كيف تتفاعل الأجزاء مع بعضها	يحدد مدى قابلية الكود للقراءة والفهم
الأدوات والمفاهيم	Layers, Dependencies, Interfaces, DI	Naming, Functions, Comments, DRY, KISS
يمكن تطبيقه منفرداً؟	نعم — لكن بدون Clean Code ستكون الطبقات مبهمة	نعم — لكن بدون Architecture الكود سيكون فوضوياً هيكلياً

٢ - العلاقة بينهما

🔗 Clean Architecture يحدد أين تضع الكود — Clean Code يحدد كيف تكتبه. هما مكمّلان لبعضهما وليسا بديلَين.

تصوّر المشروع كمبنى:

- Clean Architecture = الهيكل الإنشائي للمبنى: الأعمدة، الأدوار، توزيع الغرف.
- Clean Code = جودة البناء في كل غرفة: التشطيبات، الأسلاك، السباكة.

يمكنك الحصول على:

- Clean Architecture بدون Clean Code: مشروع منظم هيكلياً لكن الكود داخله صعب القراءة.
- Clean Code بدون Clean Architecture: كل ملف مكتوب بشكل جميل لكن الملفات غير منظمة.
- كليهما معاً: المستوى الاحترافي الحقيقي الذي نسعى إليه.

٣ - مثال عملي مقارن: Load Products

❌ الطريقة السيئة — لا Architecture ولا Clean Code

```
// كل شيء في ملف واحد — UI + Business Logic + API call
class ProductScreen extends StatefulWidget {
  @override State createState() => _S();
}
class _S extends State<ProductScreen> {
  List d = [];
  // أسماء مبهمة، لا يمكن اختباره، مدمج مع UI
  Future load() async {
```

```
    var r = await http.get(Uri.parse('http://api.com/products'));
    setState(() { d = jsonDecode(r.body); });
  }
  @override Widget build(BuildContext ctx) { ... }
}
```

❌ مشاكل هذا الكود: Business Logic مدمجة في الـ UI — أسماء مبهمة (d, r, _S) — لا يمكن اختباره بدون تشغيل الـ App — أي تغيير في الـ API سيكسر الـ UI.

✅ الطريقة الصحيحة — Clean Architecture + Clean Code

```
// Domain Layer — Entity
class Product {
  final String id;
  final String name;
  final double price;
  bool get isOnSale => price < 50.0;
}

// Domain Layer — Use Case (قابل للاختبار بشكل كامل)
class GetProductsUseCase {
  final ProductRepository repository;
  Future<List<Product>> call() => repository.getProducts();
}

// Presentation Layer — Cubit (يربط UI بـ Use Cases)
class ProductsCubit extends Cubit<ProductsState> {
  final GetProductsUseCase getProductsUseCase;
  ProductsCubit(this.getProductsUseCase) : super(ProductsInitial());

  Future<void> loadProducts() async {
    emit(ProductsLoading());
    final products = await getProductsUseCase();
    emit(ProductsLoaded(products));
  }
}
```

✅ الفرق: كل طبقة لها مسؤولية واحدة — الأسماء واضحة ومعبّرة — يمكن اختبار الـ Use Case بشكل كامل مستقل عن الـ UI والـ API.

الجزء الرابع: Checklists للمراجعة

Clean Architecture Checklist ✅

1. هل كل Feature لها مجلدها الخاص بـ domain / data / presentation؟
2. هل الـ Entities لا تحتوي على أي import خارجي؟
3. هل الـ Use Cases تعتمد على Repository Interfaces وليس Implementations؟
4. هل الـ Widgets (UI) لا تعرف شيئاً عن الـ API أو الـ Database؟
5. هل يمكن اختبار الـ Use Cases بدون تشغيل الـ App؟
6. هل الـ Dependency Injection منظم في مكان واحد؟

Clean Code Checklist ✅

7. هل كل دالة تفعل شيئاً واحداً فقط (Single Responsibility)؟
8. هل أسماء المتغيرات والدوال واضحة وذات معنى؟
9. هل عدد بارامترات الدوال 3 أو أقل؟
10. هل تم حذف الكود المعلّق بدلاً من إبقائه كتعليق؟
11. هل الأخطاء معالجة بشكل صريح ومُعبّر؟
12. هل لا يوجد تكرار للـ Business Logic في أكثر من مكان (DRY)؟

📌 ملخص نهائي

السؤال	الإجابة
ما هي Clean Architecture؟	نهج لتنظيم المشروع في طبقات مستقلة تتبع Dependency Rule
ما هو Clean Code؟	مبادئ لكتابة كود مقروء وبسيط وقابل للاختبار على مستوى الكود
ما الفرق؟	Architecture = هيكل المشروع — Code = جودة كتابة الكود
هل هما متعارضان؟	لا — هما مكمّلان لبعضهما، الاحترافية تتطلب كليهما
من أين أبدأ؟	ابدأ بـ Clean Code في كل سطر تكتبه، ثم طبّق Architecture في المشاريع الكبيرة

🚀 "The only way to go fast, is to go well." — Robert C. Martin الطريقة الوحيدة للعمل بسرعة هي العمل بشكل صحيح من البداية.